
The Cost of Saying Hello: Measuring Handshake Overhead in AI Agent Communication Protocols

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Every interaction between AI agents begins with a handshake: the initiator must
2 locate its counterpart, learn what it can do, and establish a session before the
3 first useful message. Surveys of agent communication protocols name handshake
4 overhead as an efficiency criterion, yet none report measurements. We decompose
5 the handshake into four phases and measure it in three protocols using their official
6 SDKs: MCP (in both its 2025 *initialize* and 2026 *discover* eras), A2A, and ACP.
7 Round-trip count dominates. A legacy MCP handshake costs 230 ms at 50 ms RTT
8 against 61.0 ms for A2A, and the legacy client opens a fresh TCP connection per
9 JSON-RPC exchange, a behaviour we traced to the SDK and reported upstream.
10 Round trips are not the whole bill: when an LLM reads the counterpart’s metadata,
11 cost scales with inventory size at 24–107 tokens per capability, so a 200-capability
12 counterpart spends 11% of a 200k-token context window before the agent reads its
13 own task. MCP’s own 2026 revision cuts three round trips to two, and to zero on
14 pinned warm reconnects, protocol evolution that corroborates the cost we measure.
15 We release the harness, raw per-run data, and analysis.

16 1 Introduction

17 Agents built on large language models increasingly talk to other agents: a coding assistant calls a
18 tool server, an orchestrator delegates to a specialist, a consumer agent queries a merchant. Each
19 interaction is preceded by an exchange in which the initiator discovers its counterpart, learns its
20 capabilities, and establishes a session. We call this the *handshake*. It is pure overhead. No useful
21 work happens until it completes, and in open ecosystems, where counterparts are strangers and
22 interactions are often short, its cost is paid frequently and amortised poorly.

23 The systems community has measured exactly this tax at lower layers for decades. The TCP three-
24 way handshake, the TLS key exchange [9, 10], and QUIC’s 0-RTT resumption [6] were all designed
25 and redesigned around one insight: connection establishment is a first-order cost of short interac-
26 tions. The agent protocol stack has not received the same treatment. The most comprehensive
27 survey of the space names “handshake overhead” as an explicit efficiency criterion, yet reports no
28 measurements [11], and recent agent benchmarks measure end-to-end at the task level [3, 4] without
29 isolating connection establishment.

30 We fill that gap for the Model Context Protocol (MCP) [2, 8], the Agent2Agent protocol (A2A) [1],
31 and the Agent Communication Protocol (ACP) [5], using each project’s official Python SDK against
32 a minimal counterpart built with its official server SDK. ACP merged into A2A in 2025 and is frozen;
33 we include it because deployments persist and its one-round-trip REST design is a distinct point in
34 the design space. For MCP we measure two eras separately: the *legacy* handshake (specification
35 revisions 2024-11 through 2025-11), which nearly all deployed MCP software used at the time of
36 writing, and the *modern* handshake of revision 2026-07-28, which replaces three-message initialisa-

Table 1: Task-ready definitions and observed cold-start wire sequences. Sizes are HTTP body bytes from instrumented runs.

Protocol	Task-ready condition	Observed wire sequence (cold)
MCP (legacy)	<code>initialize</code> response received; <code>initialized</code> sent; <code>tools/list</code> parsed; echo present	POST <code>initialize</code> (163 B→379 B); POST <code>initialized</code> (54 B, 202); POST <code>tools/list</code> (68 B→399 B); 3 round trips
MCP (modern)	<code>server/discover</code> adopted; <code>tools/list</code> parsed; echo present	POST <code>server/discover</code> (245 B→431 B); POST <code>tools/list</code> (240 B→522 B); 2 round trips
A2A	Agent Card fetched and parsed; skill echo present	GET <code>/well-known/agent-card.json</code> (→443 B); 1 round trip
ACP	Manifest fetched and parsed; agent echo-agent present	GET <code>/agents</code> (→507 B); 1 round trip

tion with one cacheable probe. That within-protocol comparison turns a recent design change into a natural experiment on the cost of round trips.

Our contributions are (C1) a phase decomposition of the agent handshake with a precise per-protocol definition of *task-ready* (Section 2); (C2) measurements of round trips, per-phase latency, connections, wire bytes, capability-metadata context cost, and failure behaviour across four scenario families and two network conditions, $N=50$ runs per cell (Section 4); and (C3) an open harness with one-command reproduction.

Three findings stand out. Round-trip count dominates the moment a real network is involved, and the fitted slope of latency against RTT recovers each protocol’s round-trip count almost exactly. The legacy MCP client opens a fresh TCP connection for every JSON-RPC exchange, which we verified is client behaviour rather than a measurement artifact and reported upstream. And the one cost that round-trip reduction does not touch is context: metadata scales with the counterpart’s capability inventory, and the per-capability slope differs across protocols by more than a factor of four.

2 Defining the Agent Handshake

We define the *handshake* as all communication from first contact until the initiator can send its first substantive task message with justified confidence that the counterpart can handle it, and decompose it into four phases: *discovery* (locating the counterpart and retrieving its self-describing metadata), *capability exchange* (learning what it can do), *auth/session* (establishing identity and session state), and *readiness* (residual signalling before the first task message is valid). A phase may be empty, and one message may serve several; we attribute each exchange to its primary function.

The handshake ends at *task-ready*. Because “ready” is protocol-specific we define it operationally per protocol (Table 1) against a fixed canonical capability: an echo operation returning its input verbatim. The initiator is task-ready once it has positively confirmed from protocol metadata that the counterpart offers echo. A trivial capability keeps execution cost negligible so the handshake dominates; we run one echo call after each timed handshake, outside the timed region, to confirm the ready state is real.

MCP has no in-protocol discovery, as the endpoint URL is assumed known, itself a design choice. Its legacy handshake spends one exchange on session establishment (`initialize`, which also negotiates version and capabilities), one on readiness (`initialized`), and one on capability exchange (`tools/list`). Modern MCP’s `server/discover` negotiates without creating a session. A2A and ACP fold discovery and capability exchange into a single metadata fetch, with confirmation a local parse. We exclude everything below the application layer, task execution, and teardown, but we do count TCP connections, since each implies an additional transport round trip on a real network.

3 Method

All three SDKs use `httpx`, giving a uniform instrumentation point: a recording transport wraps every request with monotonic timestamps, body sizes, and reconstructed wire sizes, tagged with a

Table 2: Main results. Medians over $N=50$ runs per cell. RT = application-layer round trips to task-ready (to rejection for S3); Conn = TCP connections opened; wire bytes are relay-counted through task-ready (localhost).

Scenario	Protocol	RT	Conn	localhost (ms)		50 ms RTT (ms)		wire bytes	
				med	p95	med	p95	up	down
S1 cold start	MCP (legacy)	3	3	6.1	8.0	230	236	1147	1522
	MCP (modern)	2	1	4.4	5.2	122	125	1070	1205
	A2A	1	1	2.8	3.0	61.0	61.9	170	569
	ACP	1	1	2.8	3.2	62.1	63.1	149	633
S2 warm repeat	MCP (legacy)	3	3	6.1	6.5	230	234	1147	1522
	MCP (modern, auto)	1	1	3.6	4.2	63.5	65.4	540	557
	MCP (modern, pinned)	0	0	2.1	2.6	6.6	7.3	0	0
	A2A	0	0	2.0	2.7	2.0	2.7	0	0
	ACP	0	0	2.1	2.5	6.6	7.4	0	0
S3 mismatch	MCP (legacy)	3	3	6.2	6.9	231	235	1147	1522
	MCP (modern)	2	1	4.4	5.5	117	123	1070	1205
	A2A	1	1	2.9	4.3	62.2	65.3	170	569
	ACP	1	1	2.9	4.0	57.7	63.2	149	633

handshake phase. Traffic additionally crosses a local TCP relay that counts exact bytes and connections. Network conditions are emulated in that relay: the *50 ms RTT* condition delays each forwarded chunk by 25 ms per direction; *localhost* uses the same relay with zero delay. We also sweep emulated RTT over $\{0, 25, 50, 100\}$ ms for the cold-start scenario.

The relay delays application-layer chunks rather than packets, which is the main threat to validity and we state it plainly: TCP setup against the relay completes at loopback speed, so our 50 ms figures *exclude* transport setup, and SSE-framed responses cross in two chunks and pay the per-chunk delay twice. Both effects are quantified in Appendix A; neither changes the ordering of protocols, and the excluded transport setup penalises exactly the protocol that opens the most connections, so our headline gap is conservative.

Scenarios are S1 cold start (no prior contact), S2 warm repeat (each protocol allowed exactly the reuse its specification and SDK support), S3 capability mismatch (measuring time-to-rejection), and S4 degraded conditions (slow counterpart, truncated metadata, version mismatch). Each run is fully cold on the client side; five warmup runs precede each cell and are discarded. We report medians and 95th percentiles over $N=50$ runs, never bare means. Every median carries a percentile bootstrap 95% confidence interval over 2,000 resamples: the median CI half-width is 0.16 ms and the largest 2.0 ms, so quoted precision is resolved by the data. Across cells, client setup drifts by a few milliseconds over a long session, so we treat cross-cell differences below roughly 5 ms as a noise floor rather than protocol effects. Every number in this paper is a generated macro computed from the released per-run logs, not a hand-typed value.

4 Results

Cold start. The protocols reach task-ready in three (MCP (legacy)), two (MCP (modern)), and one (A2A, ACP) round trips (Table 2). On loopback the differences are small in absolute terms, because a round trip costs microseconds and client-side protocol-stack work dominates. Under 50 ms RTT the spread widens to what the round-trip counts predict: 230 ms for MCP (legacy) against 122 ms for MCP (modern), 61.0 ms for A2A, and 62.1 ms for ACP, a factor of 3.8 between the extremes.

Two details matter. First, MCP (legacy) exceeds a naive 3×50 ms prediction because two of its three responses are SSE-framed and cross the relay in two delayed chunks. Second, the legacy MCP client opens 3 TCP connections, one per JSON-RPC exchange, against one each for modern MCP, A2A, and ACP. We verified this is client behaviour, not a harness artifact: the same server reuses a single keep-alive connection under raw sequential HTTP requests and under the modern-era client, with or without the relay in the path. The mechanism is visible in the SDK, where legacy-era responses are SSE-framed and the client closes each stream as soon as the JSON-RPC reply arrives, so an undrained connection cannot return to the pool. We found no client-side configuration that restores

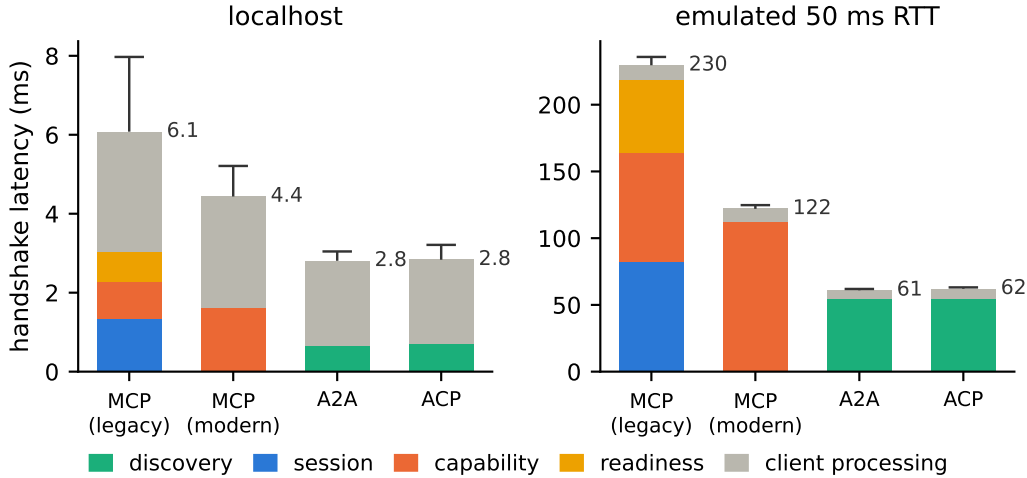


Figure 1: S1 handshake latency decomposed by phase (median stacked, whisker to p95 of the total). Note the differing y-scales: on loopback the cost is client processing; at 50 ms RTT the wire phases dominate in proportion to their round trips.

reuse in the legacy era, and reported it upstream with a deterministic reproduction.¹ Our relay hides the cost: on a real 50 ms link those extra connections would add roughly another 100 ms, and more with TLS.

The cost is round trips, not bytes or compute. The phase decomposition (Figure 1) locates it. On loopback most of every handshake is client processing: SDK object construction, JSON parsing, validation. At 50 ms RTT the picture inverts, and each phase’s share tracks its round trips. Sweeping emulated RTT makes the mechanism direct: latency rises linearly with fitted slopes of 1.1 (A2A), 1.1 (ACP), 2.1 (MCP (modern)), and 4.2 (MCP (legacy)) milliseconds per millisecond of RTT. The single-fetch protocols track their one round trip almost exactly; the legacy slope reflects three delayed requests plus five delayed response chunks, or four RTT equivalents. Wire volume tells a consistent but secondary story: 1147+1522 B for MCP (legacy) against 170+569 B for A2A, a difference invisible next to the round-trip effect at interactive timescales.

Warm reconnection separates the protocols more than cold start. MCP (legacy) can skip nothing: its specification predates response caching, so a reconnecting client repeats all three round trips (230 ms at 50 ms RTT, within noise of its cold handshake). MCP (modern) under the SDK’s default policy re-probes but serves the tool list from the SEP-2549 cache (63.5 ms). A client that additionally pins the counterpart’s protocol era reaches task-ready with zero wire exchanges in 6.6 ms, the same regime as A2A’s cached card (2.0 ms) and ACP’s cached manifest (6.6 ms). Over $k=100$ interactions with the same counterpart the per-interaction cost amortises to 230 ms for MCP (legacy), effectively unamortised, against 2.6 ms for A2A. For agent pairs that interact repeatedly, cacheability rather than cold-start cost is the design property that matters; for true strangers, the cold numbers stand.

Rejection costs as much as acceptance. Time-to-rejection mirrors time-to-ready, because in all three protocols the initiator discovers what a counterpart *cannot* do the same way it discovers what it can. A2A and ACP reject after their single metadata fetch (62.2 and 57.7 ms at 50 ms RTT), while MCP (legacy) must complete its entire three-round-trip handshake, session included, before learning the counterpart is useless to it (231 ms). Document-centric discovery makes ruling a counterpart out as cheap as one GET; session-first designs charge the session price merely to browse, which compounds for agents that shop across many counterparts.

¹ Issue reference withheld for double-blind review. ANONYMISED.

Table 3: Handshake metadata in tokens as the counterpart’s capability inventory grows, and the marginal cost of one capability. Built by replicating the captured wire entry n times inside the byte-identical envelope, so these are the documents an n -capability counterpart serves.

Protocol	$n=1$	$n=10$	$n=50$	$n=200$	per cap.
MCP	96	798	3,918	15,618	78
A2A	115	331	1,291	4,891	24
ACP	112	1,075	5,355	21,405	107

Only MCP validates version claims during the handshake. Both MCP eras refuse a counterpart advertising only a fictitious version in 3.7–4.2 ms with an explicit error, the modern probe’s structured error even carrying the server’s supported-version list. A2A’s client instead fetched a card claiming protocol version 99.0.0, selected a binding, and completed tasks without warning. This is spec-conformant, not an SDK bug: the normative schema marks the interface’s `protocolVersion` as descriptive, and A2A’s mandatory negotiation happens per request via an `A2A-Version` header the server must reject when unsupported [1]. On the wire we observed exactly that split, the client sending its own version independent of the card’s claim. The consequence for handshake design stands: discovery-time version claims go unvalidated, so a real incompatibility surfaces at the first task message rather than during the handshake, and never at all when the wire versions happen to agree. ACP has no version field to check. All protocols detect truncated metadata quickly and abort rather than degrade (Appendix B).

Context, not dollars, is the scarce resource. When the initiator is an LLM the counterpart’s metadata is not merely transferred but *read*, placed in a context window to select a tool or validate a counterpart. For a one-capability counterpart the cost is trivial, 112–242 tokens, and should be treated as a protocol floor rather than a cost. The slope is the finding (Table 3). Each additional capability costs 24 tokens in an A2A card, 78 in an MCP tool listing, and 107 in an ACP manifest. At 200 capabilities the handshake alone delivers 15,618 tokens for MCP and 21,405 for ACP, which is 11% of a 200k-token window spent before the agent reads its own task.

The per-capability costs differ by more than a factor of four, and the ordering inverts against the one-capability ranking: A2A is cheapest because a skill entry is a name, a description, and tags, MCP dearer because a tool entry carries full input and output JSON schemas, ACP dearest because each agent entry repeats a sixteen-field metadata block. A protocol’s floor says little about what it costs at deployment scale. This is also the one handshake cost that round-trip reduction does not touch: it is carried by the listing step, which every protocol performs and which the 2026 MCP revision made cacheable but no cheaper.

5 Discussion

Handshake cost is round trips first, connections second, bytes a distant third, and compute negligible. Every design question about agent handshakes should therefore be asked as “how many round trips does this add, and can it be cached?” The protocols’ own trajectories agree: MCP’s 2026 revision is precisely a round-trip-reduction release, replacing a three-message session-first initialisation with a one-probe cacheable discovery, and our measurements quantify the effect at 47% off the cold handshake at 50 ms RTT. That the youngest protocol in our study already shipped this change is evidence that handshake overhead is a live design pressure rather than a hypothetical.

For an OS layer over agentic systems, three implications follow directly. Treat counterpart metadata as a cacheable asset with a lifecycle, since the measured gap between MCP (legacy) and everything else is mostly the gap between re-deriving and reusing handshake state. Budget round trips at discovery time when an agent shops across many counterparts, because rejection costs a full handshake and a design that rules counterparts out from one GET scales shopping linearly rather than in handshakes. And treat the context window as a first-class budget: the per-capability slope, not the floor, is what binds, and it is the cost caching defers but round-trip reduction never removes.

The measurements have real limits. Our counterparts expose one capability, so latency figures are protocol floors, which is why we sweep inventory size for the metadata costs. Emulation is userspace rather than kernel level, so transport setup is excluded from the RTT figures, and we quantify the

direction of that bias rather than claim it away. We measure specific SDK versions at a specific date, and SDK-added behaviour is part of what we measure deliberately, because it is what deployed agents actually pay. Authenticated handshakes and live natural-language negotiation are the obvious next cells; both add round trips and tokens that deserve the same decomposition.

References

- [1] A2A Project, Linux Foundation. Agent2Agent (A2A) protocol specification, v1.0.0. <https://a2a-protocol.org/v1.0.0/specification>, 2026. Released 2026-03-12; accessed 2026-08-10.
- [2] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Published 2024-11-25.
- [3] Ivan Dobrovolskyi. Empirical comparison of agent communication protocols for task orchestration, 2026. URL <https://arxiv.org/abs/2603.22823>.
- [4] Hongyi Du, Jiaqi Su, Jisen Li, Lijie Ding, Yingxuan Yang, Peixuan Han, Xiangru Tang, Kunlun Zhu, and Jiaxuan You. ProtocolBench: Which LLM multi-agent protocol to choose? In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2510.17149>. To appear; arXiv:2510.17149.
- [5] IBM BeeAI project. Agent Communication Protocol specification. <https://agentcommunicationprotocol.dev/>, 2025. Version 1.0; repository archived 2025-08-27; accessed 2026-08-10.
- [6] Jana Iyengar and Martin Thomson. QUIC: A UDP-based multiplexed and secure transport. Technical Report RFC 9000, IETF, 2021.
- [7] Hyunwoo Lee, Doowon Kim, and Yonghwi Kwon. TLS 1.3 in practice: How TLS 1.3 contributes to the internet. In *Proceedings of the Web Conference (WWW)*, 2021. doi: 10.1145/3442381.3450057.
- [8] Model Context Protocol project. Model Context Protocol specification, revision 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026. Accessed 2026-08-10.
- [9] David Naylor, Alessandro Finamore, Ilias Leontiadis, Yan Grunenberger, Marco Mellia, Maurizio Munafò, Konstantina Papagiannaki, and Peter Steenkiste. The cost of the “s” in HTTPS. In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 133–140, 2014. doi: 10.1145/2674005.2674991.
- [10] Eric Rescorla. The Transport Layer Security (TLS) protocol version 1.3. Technical Report RFC 8446, IETF, 2018.
- [11] Yingxuan Yang, Huacan Chai, Yuanyi Song, Siyuan Qi, Muning Wen, Ning Li, Junwei Liao, Haoyi Hu, Jianghao Lin, Gaowei Chang, Weiwen Liu, Ying Wen, Yong Yu, and Weinan Zhang. A survey of AI agent protocols, 2025. URL <https://arxiv.org/abs/2504.16736>.

A Threats to Validity

Toy counterparts. Our agents expose one capability. Real counterparts with large inventories shift byte and token costs upward, which is why Table 3 sweeps inventory size directly; handshake latency is less affected, since inventory changes payloads rather than round trips.

Userspace network emulation. The relay delays application-layer chunks, not packets. TCP establishment against it completes at loopback speed, so the 50 ms-RTT figures exclude transport setup entirely; on a real 50 ms link each connection would add one RTT, plus one for TLS 1.3, which penalises exactly the protocols that open more connections. SSE responses cross in two chunks and

225 pay the per-chunk delay twice, which parallels but does not perfectly model per-packet kernel emula-
 226 tion. Both biases run against the protocol we report as cheapest, so the measured gap is conservative.
 227 Reproducing these cells under kernel-level emulation is the first thing we would add.

228 *Single SDK generation, single machine.* We measure specific SDK versions on one Apple-silicon
 229 machine over loopback. Round-trip and byte counts, which drive the higher-RTT results, are
 230 machine-independent; absolute low-RTT latencies are not.

231 *Token accounting.* Token counts use an open tokenizer as a lower bound; a frontier vendor’s tok-
 232 enizer counts roughly 15–20% higher on JSON-like content. Byte counts are exact.

233 B Degraded Conditions in Full

Table 4: Behaviour under degraded conditions ($N=50$ per cell, localhost). S4a reports time to task-ready with a 2 s per-response server delay; S4b and S4c report time to failure. For S4c, ACP is unstable: its manifest carries no protocol-version field.

Scenario	Protocol	Outcome	med. time (ms)	Retr.	Error surfaced
S4a slow (2 s/resp.)	MCP (legacy)	ready	6026	0	none
	MCP (modern)	ready	4019	0	none
	A2A	ready	2011	0	none
	ACP	ready	2011	0	none
S4b truncated metadata	MCP (legacy)	error	3.5	0	MCPErrors
	MCP (modern)	error	4.4	0	MCPErrors
	A2A	error	3.0	0	AgentCardResolutionError
	ACP	error	2.9	0	JSONDecodeError
S4c version mismatch	MCP (legacy)	error	3.7	0	RuntimeError
	MCP (modern)	error	4.2	0	RuntimeError
	A2A	ready	3.0	0	none (version deferred)

234 With 2 s injected per response, time to task-ready is a clean multiple of round-trip count: 6026 ms
 235 for MCP (legacy) against about 2 s for A2A and ACP. No SDK timed out or retried; the delay simply
 236 multiplies, so a slow counterpart is most expensive to the protocol with the most round trips, and the
 237 initiator cannot discover the slowness before paying the full handshake.

238 Against metadata truncated at 50%, all four variants fail fast, with no retry storms and no hangs,
 239 every client detecting invalid JSON on first parse in milliseconds. What differs is error quality.
 240 A2A names the offending URL and MCP reports a protocol-level validation failure, while ACP
 241 surfaces a bare JSONDecodeError with a character offset and no indication of which endpoint
 242 or agent was involved. All three abort rather than degrade, which is right; only ACP leaves the
 243 caller to reconstruct the context. For autonomous agents, which cannot eyeball a stack trace, the
 244 difference between a structured “unsupported version, here is what I support” and a counterpart
 245 whose incompatibility surfaces only at task time is the difference between a recoverable negotiation
 246 and a debugging session.

247 C Transport-Layer Baseline

248 For calibration, on the same loopback a bare TCP connect completes in 0.05 ms and a full TLS 1.3
 249 handshake in 1.88 ms of compute, one round trip each on a real network [10]. Against that two-
 250 round-trip transport-plus-cryptographic stack, A2A and ACP add a single application-layer round
 251 trip, half the cost of the stack beneath them, while a legacy MCP handshake adds three, exceeding the
 252 combined transport and cryptographic setup it rides on, before any authentication. The application
 253 layer has in effect re-created the pre-TLS-1.3 handshake stack one level up, and is now re-learning
 254 the same remedies, fewer round trips and resumable state, that transport designers adopted a decade
 255 ago [6, 7].

256 **D Reproducibility**

257 The released repository contains the harness, minimal counterpart agents, raw per-run JSONL for
258 every cell in this paper, and the analysis scripts that generate every figure, table, and in-text num-
259 ber. Three commands reproduce the raw data and one regenerates every artifact from it; the first
260 takes roughly 25 minutes on a laptop. Version pins: Python 3.12.13, mcp 2.0.0, a2a-sdk 1.1.2,
261 acp-sdk 1.0.3, httpx 0.28.1, uvicorn 0.34.3, fastapi 0.115.14, starlette 0.46.2. The MCP
262 counterpart is served over streamable HTTP with SEP-2549 cache hints (one hour, public scope)
263 on `server/discover` and `tools/list`; the A2A counterpart serves a v1.0 Agent Card with one
264 skill over the JSON-RPC binding; the ACP counterpart serves one agent manifest. Repository URL
265 withheld for double-blind review. ANONYMISED.